INTERNSHIP PROJECT DOCUMENTATION

DESIGN AND IMPLEMENTATION OF AN AES-128 CRYPTOGRAPHIC PROCESSOR USING VERILOG HDL

Submitted by

Name: Syed Azam Ali

Roll Number: 2451-23-735-058

Department: Electronics and Communication Engineering

College: MVSR College of Engineering

Internship Organization: LEARNSQUARE Technologies

Internship Duration: 25 March 2026 – 22 June 2026

Internship Project Report

CERTIFICATE OF COMPLETION

ACKNOWLEDGEMENT

I want to express my sincere gratitude to LearnSquare Technologies for giving me the opportunity to undertake this internship and work on a practical digital design project involving Verilog HDL and AES-128 cryptography.

I am thankful to the mentors and coordinators who provided guidance throughout the internship, helped clarify technical concepts, and encouraged a structured approach to RTL design, simulation, and verification.

I also express my gratitude to the faculty members of the Department of Electronics and Communication Engineering, MVSR College of Engineering, for their academic support and for encouraging practical application of engineering concepts.

Finally, I thank my family and peers for their continuous support during the internship and project development period.

SYED AZAM ALI

2451-23-735-058

TABLE OF CONTENTS:

1. Abstract
2. Executive Summary
3. Introduction
4. Company/Organization Profile
5. Internship Activities
6. Technical Details
7. Learning Outcomes
8. Conclusion
9. References
10. Appendices

1. ABSTRACT

This internship project presents the design and functional verification of an AES-128 cryptographic processor using Verilog Hardware Description Language (HDL). The project focuses on translating the operations of the Advanced Encryption Standard (AES) into synthesizable-style RTL modules suitable for hardware implementation. The processor accepts a 128-bit plaintext block and a 128-bit secret key and produces a 128-bit ciphertext after performing the AES-128 transformation sequence.

The implementation was developed as a modular RTL design. Major functional elements include the AES S-Box for byte substitution, ShiftRows transformation, MixColumns transformation, AddRoundKey operation, key expansion logic for round-key generation, and a finite-state-machine-based controller for sequencing the encryption process. Individual modules were developed and verified before being integrated into the top-level processor.

Functional verification was performed using Verilog testbench logic in an online HDL simulation environment. A known AES-128 test vector was applied to the processor. The generated ciphertext matched the expected reference ciphertext, and the simulation reported an AES test pass after completion of the ten AES-128 rounds. The project provided practical experience in RTL design, modular hardware architecture, digital logic, simulation, debugging, and verification.

2. EXECUTIVE SUMMARY

The internship project was completed from 25 March 2026 to 22 June 2026 with the objective of gaining practical exposure to digital hardware design through a complete cryptographic RTL implementation. The selected application was AES-128 because it combines well-defined mathematical transformations with practical requirements for secure hardware systems.

The work began with an understanding of AES-128, its 128-bit data path, 128-bit key, ten encryption rounds, and round transformations. The algorithm was then decomposed into hardware-friendly modules. Particular attention was given to byte-level transformations and finite-field operations required by MixColumns and key expansion.

After module-level development, the blocks were connected through a controller that manages initialization, round processing, final-round processing, and completion.

Testbench-driven simulation was used to observe signals such as clock, start, done, plaintext, secret key, and ciphertext. The final test compared the RTL-generated ciphertext with the known expected value.

The final verification demonstrated functional correctness for the selected test vector. The project therefore served as an end-to-end exercise covering algorithm study, RTL coding, module integration, simulation, debugging, and result interpretation.

3. INTRODUCTION

3.1 Background

Modern electronic systems exchange and store large quantities of digital information. Security mechanisms are therefore required to protect data against unauthorized disclosure and modification. Cryptographic algorithms provide mathematical methods for transforming readable information into protected representations.

The **Advanced Encryption Standard (AES)** is a symmetric-key block cipher standardized for secure processing of digital information. AES supports block sizes of 128 bits and key sizes of 128, 192, and 256 bits. In AES-128, a 128-bit key is used and the encryption process consists of an initial key addition followed by ten rounds of transformations.

Although cryptographic algorithms are often discussed as software functions, their regular structure makes them suitable for hardware implementation. A hardware-oriented design can expose parallelism at the byte level, provide deterministic processing, and form a reusable IP block for larger digital systems.

3.2 Internship Objectives

- Understand Verilog, SystemVerilog, and Analog concepts of CMOS Transistors
- Understand the principles of AES-128 symmetric-key encryption.
- Translate cryptographic operations into modular Verilog RTL.
- Develop and integrate individual AES functional blocks.
- Implement a controller for sequencing the encryption rounds.
- Create testbench logic for functional verification.
- Use simulation waveforms and console output to debug RTL behaviour.
- Verify the final ciphertext against a known AES-128 reference vector.

3.3 Relevance to Academic Curriculum

The project directly relates to digital electronics, computer architecture, VLSI design, HDL-based modeling, finite-state machines, combinational and sequential logic, and hardware verification. It also demonstrates how an algorithm studied at the software or mathematical level can be converted into an RTL architecture for digital hardware.

3.4 Project Scope

The scope of the internship project is the RTL-level design and functional simulation of an AES-128 encryption processor. The work does not claim physical fabrication, ASIC tape-out, FPGA deployment, or measured silicon power unless separately performed. The primary outcome is a verified RTL implementation whose output agrees with a known AES-128 reference result.

4. COMPANY/ORGANIZATION PROFILE

LearnSquare Technologies was the internship organisation associated with this project. The internship was structured around practical project work and technical learning, with the project documentation requirement emphasizing the student's activities, technical contribution, tools used, outcomes, and skills acquired.

LEARNSQUARE

For this internship, the project work provided an opportunity to apply HDL and digital-design concepts in a structured engineering task. The AES-128 processor was treated as a practical RTL design problem rather than only as a theoretical cryptography exercise.

4.1 Internship Information

Parameter	Details
Student	Syed Azam Ali
Roll Number	2451-23-735-058
Organization	LearnSquare Technologies
Project	AES-128 Cryptographic Processor Using Verilog HDL
Start Date	25 March 2026
End Date	21 June 2026
Domain	Digital Design / VLSI / RTL / Hardware Cryptography

5. INTERNSHIP ACTIVITIES

5.1 Initial Study and Requirement Analysis

The initial stage involved understanding the internship project requirement and selecting AES-128 as the target cryptographic function. The AES specification, data path, round structure, and required transformations were studied before RTL coding.

5.2 RTL Module Development

The AES processor was developed using a modular approach. Instead of placing the complete algorithm in one large block, major operations were separated into reusable RTL modules. This improved readability, debugging, and the ability to verify individual transformations.

- S-Box / SubBytes implementation for nonlinear byte substitution.
- ShiftRows transformation for byte permutation.
- MixColumns transformation using finite-field arithmetic.
- Supporting multiplication operations such as xtime and multiplication by 3.
- Key expansion logic for generating round keys.
- FSM controller for sequencing initialization, regular rounds, final round, and completion.
- Top-level integration and testbench development.

5.3 Simulation and Debugging

Simulation was used throughout development to identify functional errors before integration. Intermediate outputs were examined when required, and the MixColumns implementation was specifically debugged until its verification passed. The integrated processor was then tested with a standard AES-128 plaintext/key pair.

5.4 Tools and Technologies

Tool / Technology	Purpose
Verilog HDL	RTL design of AES modules and controller
EDA Playground	Online HDL simulation and waveform inspection
Verilog Testbench	Applying inputs and checking expected output
Waveform Viewer	Observing clock, control and data signals

5.5 Challenges and Solutions

Challenge	Approach Used
Complexity of AES transformations	The algorithm was divided into smaller modules and verified incrementally.
Finite-field arithmetic in MixColumns	Helper operations such as xtime and multiplication-by-3 logic were used to implement the required byte arithmetic.
Round sequencing	An FSM controller was used to control initialization, regular rounds, final round, and completion.
Debugging integration errors	Simulation waveforms and intermediate values were examined to locate incorrect transformations.
Output verification	A known AES-128 test vector was used so the generated ciphertext could be compared directly with the expected result.

6. TECHNICAL DETAILS

6.1 AES-128 Overview

AES operates on a 128-bit block represented internally as a 4 × 4 array of bytes called the state. AES-128 uses a 128-bit cipher key and derives eleven 128-bit round keys: one for the initial AddRoundKey operation and one for each of the ten encryption rounds.

The first stage of encryption performs AddRoundKey. Rounds 1 through 9 perform SubBytes, ShiftRows, MixColumns, and AddRoundKey. Round 10 performs SubBytes, ShiftRows, and AddRoundKey, omitting MixColumns.

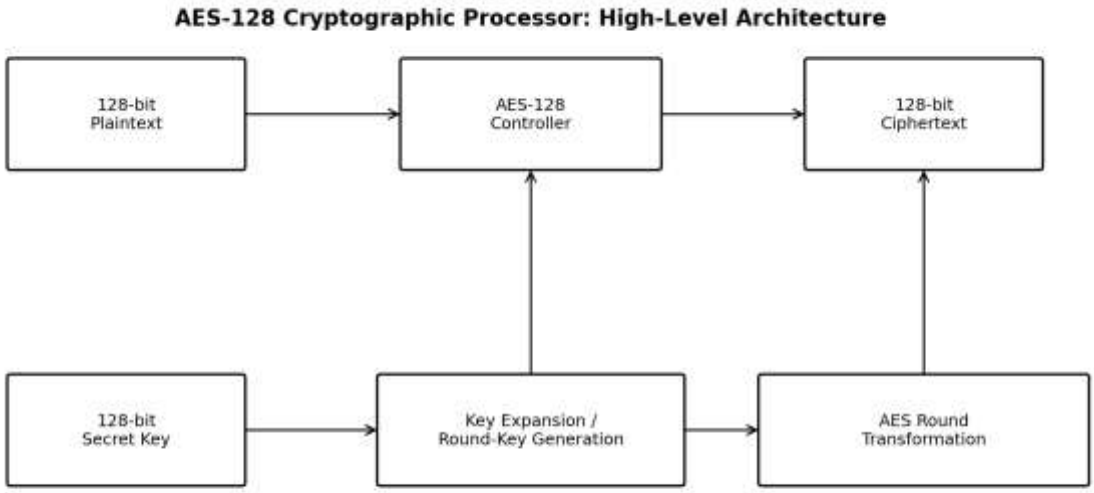


Figure 6.1: High-level architecture of the AES-128 cryptographic processor

6.2 AES Encryption Flow

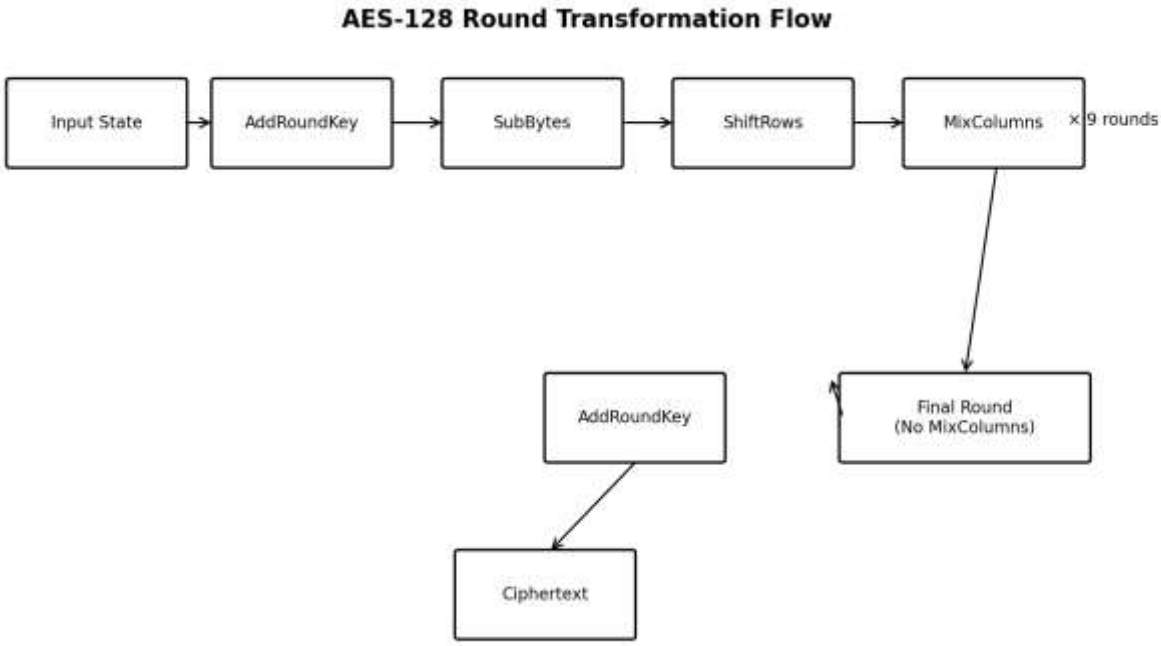


Figure 6.2: AES-128 round transformation flow

6.3 SubBytes

SubBytes replaces every byte in the AES state using a fixed 8-bit to 8-bit substitution table called the S-Box. The S-Box provides the nonlinear component of AES and is essential to the cipher's resistance to simple linear relationships between input and output.

In the RTL implementation, the substitution operation is represented as a byte-wise lookup. Since the state contains 16 bytes, the operation is applied to each byte of the 128-bit state.

6.4 ShiftRows

ShiftRows is a deterministic permutation of the state bytes. The first row remains unchanged, while the second, third, and fourth rows are cyclically shifted by one, two, and three byte positions respectively. This operation spreads the influence of individual bytes across columns in later stages.

6.5 MixColumns

MixColumns transforms each four-byte column using matrix multiplication over the finite field GF(2^8). The implementation therefore requires byte-level XOR and finite-field multiplication. Supporting operations such as xtime were used to simplify multiplication by the AES polynomial x.

During development, the MixColumns logic was independently debugged and verified before integration into the complete encryption flow. This modular verification reduced the difficulty of diagnosing errors in the final processor.

6.6 AddRoundKey

AddRoundKey combines the current 128-bit state with the corresponding 128-bit round key using bitwise XOR. It is the stage that directly incorporates the secret-key material into every AES round.

6.7 Key Expansion

AES-128 expands the original 128-bit cipher key into a sequence of round keys. The key schedule uses word rotations, S-Box substitution, round constants, and XOR operations. The resulting round keys are supplied to the encryption rounds by the processor control logic.

6.8 FSM Controller

A finite-state-machine controller coordinates the encryption sequence. The implemented control structure includes an idle condition, initialization, regular-round processing, final-round processing, and completion. The controller ensures that the datapath operations occur in the required order and that the done signal is asserted after encryption is complete.

The simulation output shows completion after ROUND=10 with DONE=1, consistent with the ten-round structure of AES-128.

6.9 Top-Level Interface

Signal	Width	Description
Clk	1 bit	Clock input
rst	1 bit	Reset control
start	1 bit	Starts an encryption transaction
plaintext	128 bits	Input data block
secretkey	128 bits	AES-128 cipher key
ciphertext	128 bits	Encrypted output block
done	1 bit	Indicates completion

6.10 Representative Top-Level RTL Structure

```
module aes_top(clk,rst,start,plaintext,secretkey,done,ciphertext);
   input clk;
    input rst;
    input start;
  input [127:0] plaintext;
  input [127:0] secretkey;
  output reg [127:0] ciphertext;
    output done;
//This module was implemented to combine all implemented blocks into a single processor
endmodule
```

The complete implementation was developed using separate RTL modules for the functional transformations and controller. The representative structure above illustrates the principal testbench-level interface visible during simulation.

6.11 Simulation and Verification Methodology

Verification was performed by applying a known plaintext and 128-bit secret key to the top-level AES processor. The testbench monitored the clock, start, done, plaintext, secret key, and ciphertext signals. After the controller completed the encryption sequence, the generated ciphertext was compared against the expected AES-128 reference value.

The verification approach can be summarized as: reset the processor, apply plaintext and key, assert start, allow the FSM to process the ten AES rounds, wait for done, and compare ciphertext with the known expected value.

6.12 Verified Test Vector

Parameter	Value
Plaintext	3243f6a8885a308d313198a2e0370734
Secret Key	2b7e151628aed2a6abf7158809cf4f3c
Generated Ciphertext	3925841d02dc09fbdc118597196a0b32
Expected Ciphertext	3925841d02dc09fbdc118597196a0b32
AES Rounds	10
Completion	DONE = 1
Verification Status	AES TEST PASSED

6.13 Simulation Waveform

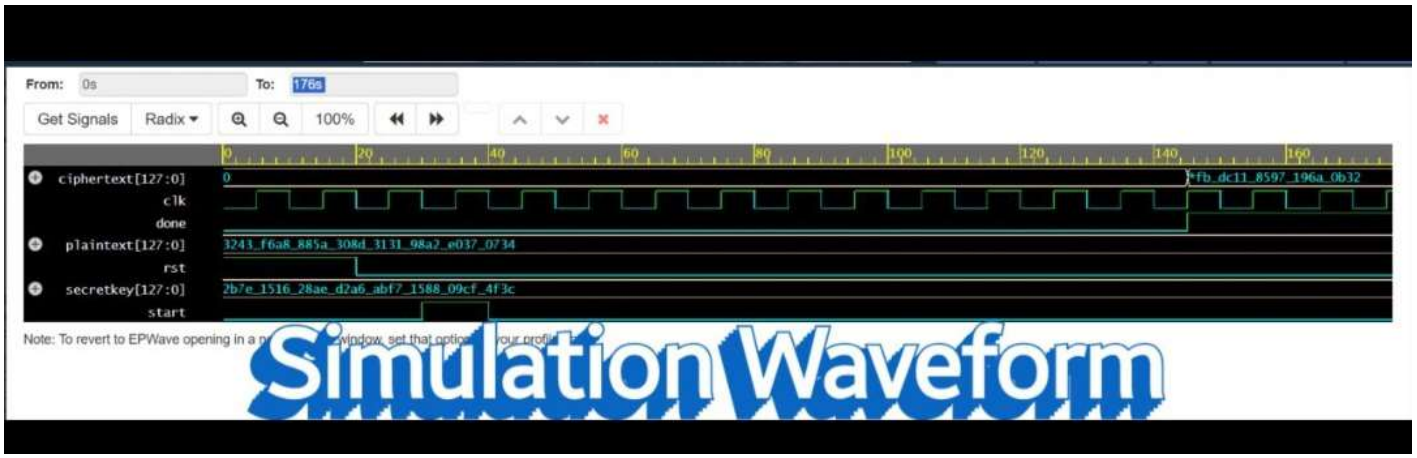
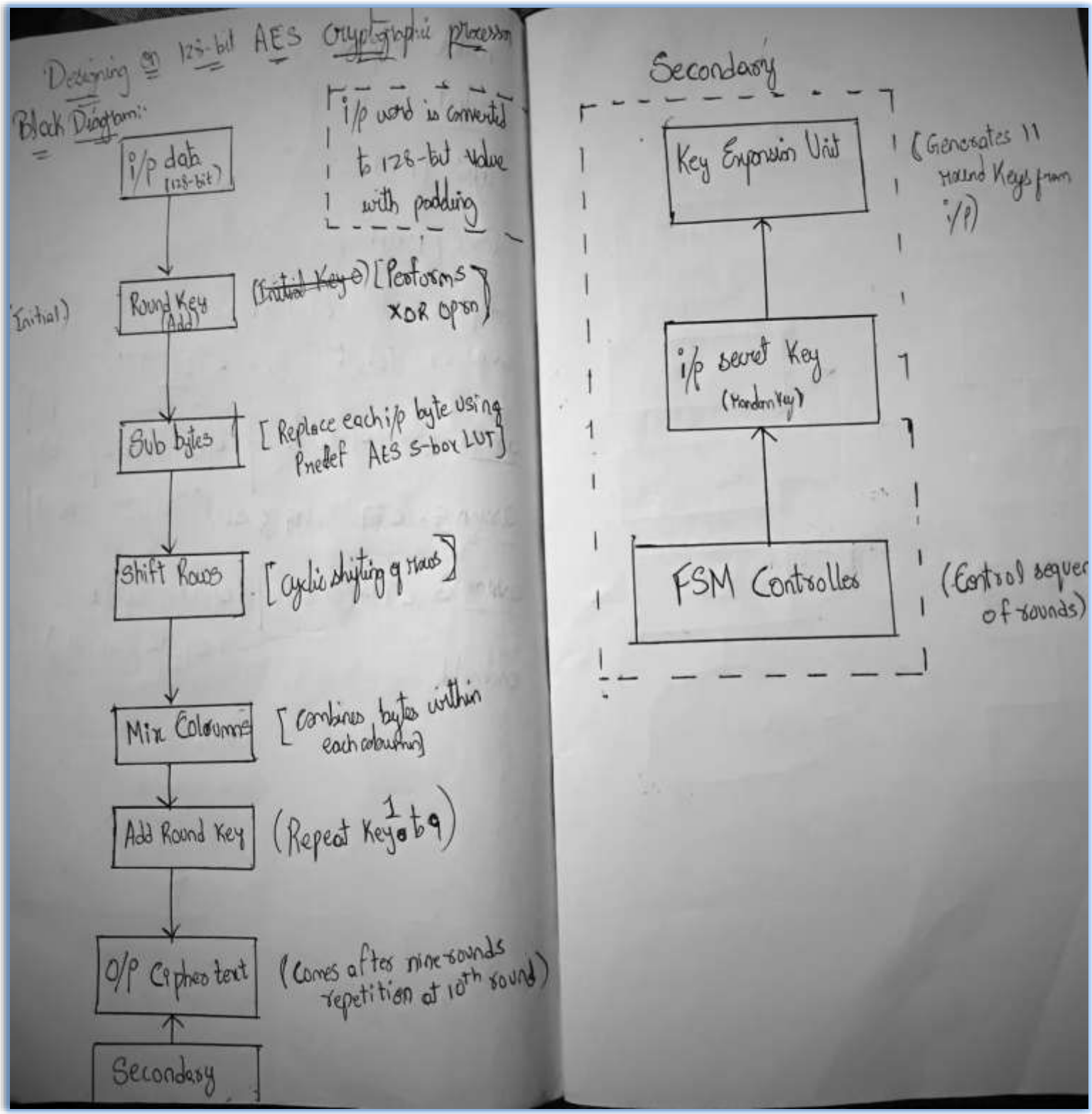
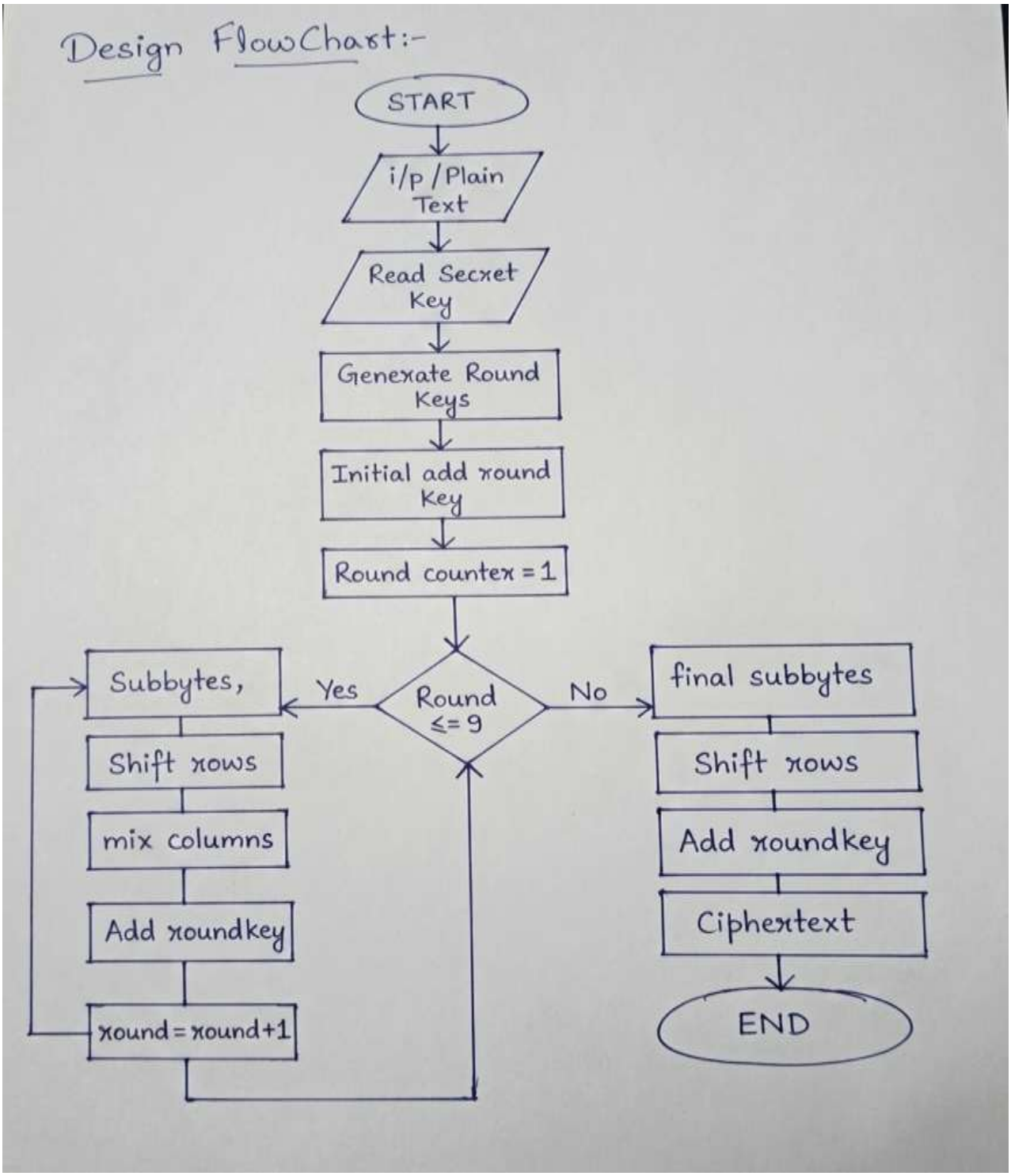


Figure 6.3: Simulation waveform showing AES processor inputs, control signals, and ciphertext

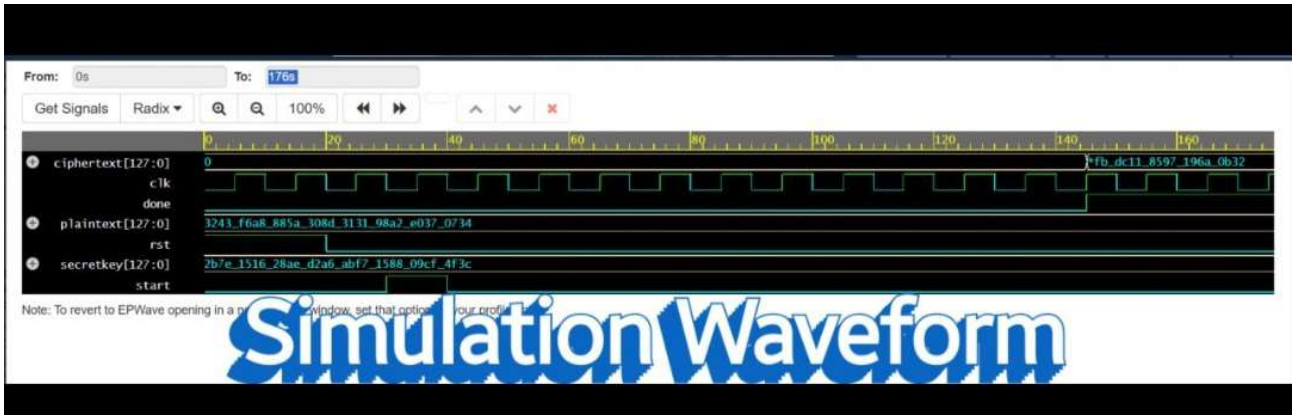


DESIGNED BLOCK DIAGRAM OF THE PROJECT



DESIGNED FLOW CHART OF THE PROJECT

6.14 Final Verification Output



The waveform demonstrates the clocked operation of the processor and the relationship between the plaintext, secret key, start, done, and ciphertext signals. It provides visual evidence that the processor performs the encryption as a sequential hardware operation.

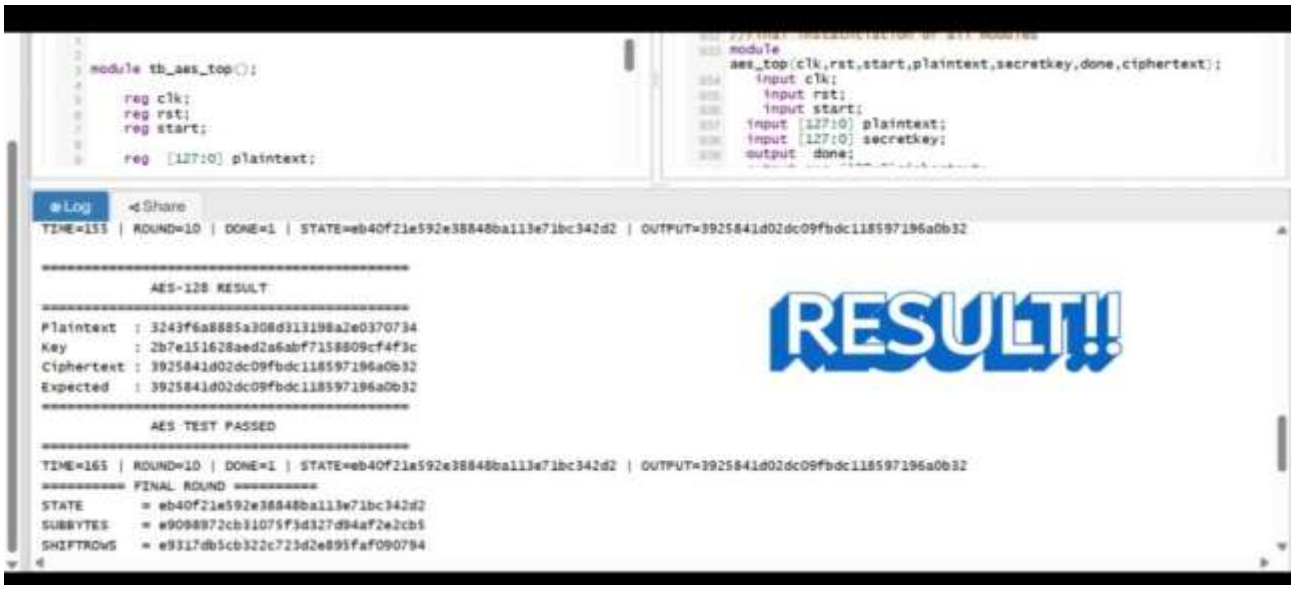


Figure 6.4: Simulation console showing AES-128 result and AES TEST PASSED

The final simulation output reports the plaintext, key, generated ciphertext, and expected ciphertext. The generated and expected ciphertext values are identical. The console also reports ROUND=10 and DONE=1, followed by the AES TEST PASSED indication. This confirms functional correctness for the selected standard test vector.

6.15 Result Discussion

The most important project result is functional agreement between the RTL implementation and the expected AES-128 output. The result confirms that the integrated datapath and control sequence correctly execute the selected encryption test. The project therefore achieved its primary objective of implementing and functionally verifying an AES-128 cryptographic processor at RTL level.

No unsupported claims are made regarding physical ASIC area, power consumption, timing closure, or FPGA resource utilization because these measurements were outside the demonstrated simulation result. Such metrics can be obtained in future work by synthesizing the RTL using an appropriate FPGA or ASIC design flow.

7. LEARNING OUTCOMES

7.1 Technical Skills Acquired

- Practical use of Verilog HDL for RTL design.
- Modular decomposition of a complex digital algorithm.
- Design of combinational byte-level transformations.
- Implementation of finite-field operations required by AES.
- Finite-state-machine based control design.
- Construction of Verilog testbenches.
- Simulation waveform analysis and RTL debugging.
- Functional verification using known reference vectors.
- Understanding of hardware-oriented cryptographic architectures.

7.2 Application of Theory to Practice

The project connected classroom concepts such as Boolean logic, combinational circuits, sequential circuits, FSMs, registers, XOR operations, and HDL modeling with a real cryptographic application. The AES algorithm provided a useful example of how mathematical transformations can be represented as hardware datapath operations and coordinated using control logic.

7.3 Personal and Professional Development

The internship strengthened structured problem solving and debugging skills. Rather than treating a failed simulation as a final result, the development process encouraged breaking the problem into smaller modules, checking intermediate behavior, and progressively integrating verified components. This workflow is directly relevant to future VLSI RTL design and verification tasks.

8. CONCLUSION

The internship project successfully demonstrated the design and functional verification of an AES-128 cryptographic processor using Verilog HDL. The implementation translated the AES-128 encryption procedure into modular hardware blocks including the S-Box, ShiftRows, MixColumns, AddRoundKey, key expansion, and an FSM-based controller.

The final RTL was verified through simulation using a known AES-128 plaintext and secret-key pair. The generated ciphertext, 3925841d02dc09fbdc118597196a0b32, matched the expected ciphertext exactly. The simulation also indicated completion of the tenth round and asserted the done signal, resulting in an AES TEST PASSED status.

Overall, the project met its intended learning and implementation objectives by providing practical experience in RTL design, cryptographic hardware, modular architecture, simulation, and functional verification. The work also establishes a foundation for future enhancement through synthesis, FPGA deployment, performance optimization, and hardware security analysis.

9. REFERENCES

[1] National Institute of Standards and Technology (NIST), FIPS PUB 197, Advanced Encryption Standard (AES), U.S. Department of Commerce.

[2] J. Daemen and V. Rijmen, The Design of Rijndael: AES – The Advanced Encryption Standard, Springer.

[3] IEEE, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language.

[4] IEEE, IEEE Standard Verilog Hardware Description Language.

[5] Verilog HDL simulation and RTL design documentation used during the internship.

[6] AES-128 standard test vector used for functional verification of the RTL implementation.

10. APPENDICES

Appendix A: Project Module Summary

Module / Block	Primary Function	Verification Focus
S-Box / SubBytes	Byte substitution	Correct lookup/output mapping
ShiftRows	State-byte permutation	Correct row shifts
MixColumns	GF(2^8) column transformation	Finite-field arithmetic
xtime / helper multiplication	Finite-field helper operations	Byte multiplication
Key Expansion	Round-key generation	Round-key sequence
FSM Controller	Encryption sequencing	Round count and done
AES Top Module	Integration of datapath and control	End-to-end ciphertext
Testbench	Stimulus and result comparison	Expected vs generated output

Appendix B: Key Verification Output

```
AES-128 RESULT

Plaintext  : 3243f6a8885a308d313198a2e0370734
Key        : 2b7e151628aed2a6abf7158809cf4f3c
Ciphertext : 3925841d02dc09fbdc118597196a0b32
Expected   : 3925841d02dc09fbdc118597196a0b32

AES TEST PASSED
ROUND = 10
DONE  = 1
```

Appendix C: Suggested Future Hardware Validation

- Synthesize the RTL and record LUT/flip-flop or gate-level utilization.
- Implement the design on an FPGA development board and verify real-time operation.
- Measure maximum operating frequency and latency after synthesis and place-and-route.
- Investigate resource sharing, pipelining, and parallel S-Box architectures.
- Study power-aware implementation and side-channel-resistant AES architectures.